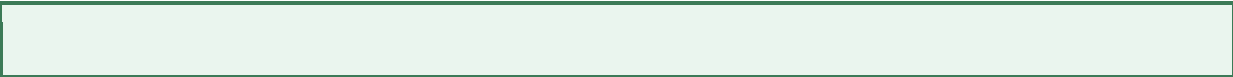


Mini Project 6. Memilih Filter Final dan Menyiapkan Artefak Firmware

Mini Project 6 melanjutkan langsung Mini Project 5. Mahasiswa tidak merekam ulang audio dan tidak mengulang eksperimen dasar FIR/IIR. Skrip membaca audio channel + hum, baseline koefisien, dan metrik MP5; mendefinisikan acceptance criteria; mengevaluasi kandidat formal; memilih filter final; mengeksport koefisien ke NPZ dan header C; membuat test vector; serta memperbarui status menjadi ready_for_mp07.



Hasil Wajib Mini Project 6

Tabel 6.10. Artefak keluaran Mini Project 6.

Artefak	Lokasi	Dipakai pada MP7
Spesifikasi desain	config/filter_design_spec.json	Menjelaskan alasan filter yang dipakai
Konfigurasi final	config/filter_final_config.json	Parameter preprocessing
Koefisien semua kandidat	features/intermediate/mp06_filter_candidates.npz	Audit desain
Koefisien final	features/intermediate/mp06_filter_final.npz	Referensi filter
Header C	firmware/mp06_filter_coefficients.h	Persiapan implementasi embedded
Test vector	features/intermediate/mp06_filter_test_vector.npz	Verifikasi perangkat
Audio final	data/processed/mp06_design/nyala_001_filter_final.wav	Input Mini Project 7
Metrik desain	results/metrics/mp06_filter_design_metrics.json	Dokumentasi keputusan
Plot kandidat	results/figures/mp06_filter_candidates.png	Validasi respons
Plot efek final	results/figures/mp06_filter_final_effect.png	Validasi audio
Status	project_status.json = ready_for_mp07	Handoff serial

Langkah 1. Periksa Handoff dari Mini Project 5

Buka terminal pada folder audio_ai_embedded_project. Pastikan file berikut tersedia: project_status.json, config/filter_experiment_config.json, features/intermediate/mp05_filter_coefficients.npz, results/metrics/mp05_filter_metrics.json, dan data/processed/mp05_filter/nyala_001_channel_hum.wav. Status wajib ready_for_mp06.

```
python -c "import json; s=json.load(open('project_status.json')); print(s['status']); assert s['status']=='ready_for_mp06'"
```

Langkah 2. Buat Skrip Mini Project 6

Buat file `mini_project_06.py` di root proyek, lalu salin kode berikut. Skrip secara sengaja menggunakan kandidat yang jumlahnya sedikit agar selesai dalam satu sesi singkat. Ia memilih filter berdasarkan acceptance criteria dan skor sederhana, bukan berdasarkan tampilan grafik.

```
from pathlib import Path
import json
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import wavfile
from scipy.signal import lfilter, firwin, remez, iirdesign, iirnotch, freqz, welch, group_delay

ROOT = Path(__file__).resolve().parent
STATUS_PATH = ROOT / "project_status.json"

# 1. Validasi handoff MP5
with open(STATUS_PATH, "r", encoding="utf-8") as f:
    status = json.load(f)
assert status.get("status") == "ready_for_mp06", "Jalankan Mini Project 5 terlebih dahulu."

input_wav = ROOT / status["filter_input_hum"]
fs, x_pcm = wavfile.read(input_wav)
if x_pcm.ndim > 1:
    x_pcm = x_pcm[:, 0]
if np.issubdtype(x_pcm.dtype, np.integer):
    x = x_pcm.astype(np.float64) / np.iinfo(x_pcm.dtype).max
else:
    x = x_pcm.astype(np.float64)

# 2. Acceptance criteria tetap untuk proyek ini
spec = {
    "sample_rate_hz": int(fs),
    "target_interference_hz": 50.0,
    "hum_attenuation_min_db": 25.0,
    "speech_band_hz": [150.0, 4000.0],
    "speech_spectral_distortion_max_db_rms": 1.5,
    "additional_delay_max_ms": 10.0,
    "multiplies_per_sample_target": 30,
    "selection_rule": "meet quality first, then minimize distortion + compute cost + delay",
}
config_dir = ROOT / "config"
feat_dir = ROOT / "features/intermediate"
out_audio = ROOT / "data/processed/mp06_design"
out_met = ROOT / "results/metrics"
out_fig = ROOT / "results/figures"
firmware = ROOT / "firmware"
for d in [config_dir, feat_dir, out_audio, out_met, out_fig, firmware]:
    d.mkdir(parents=True, exist_ok=True)
with open(config_dir / "filter_design_spec.json", "w", encoding="utf-8") as f:
    json.dump(spec, f, indent=2)

# 3. Kandidat desain formal
candidates = {}

# A. FIR window: baseline terkontrol
b = firwin(161, 115.0, pass_zero=False, window="hamming", fs=fs)
candidates["fir_hamming_hp"] = {"b": b, "a": np.array([1.0]), "type": "FIR", "delay_ms": (len(b)-1)/2/fs*1000}

# B. FIR Parks-McClellan: stopband 0-80 Hz, passband >=150 Hz
b = remez(161, [0, 80, 150, fs/2], [0, 1], weight=[10, 1], fs=fs, maxiter=1000)
candidates["fir_remez_hp"] = {"b": b, "a": np.array([1.0]), "type": "FIR", "delay_ms": (len(b)-1)/2/fs*1000}

# C. IIR Elliptic high-pass sesuai spesifikasi formal
b, a = iirdesign(wp=150, ws=80, gp=1, gstop=40, fs=fs, ftype="ellip", output="ba")
candidates["iir_ellip_hp"] = {"b": b, "a": a, "type": "IIR", "delay_ms": 0.0}

# D-E-F. Notch terlokalisasi dengan tiga Q untuk kebutuhan hum 50 Hz
for q in [20.0, 30.0, 40.0]:
    b, a = iirnotch(50.0, Q=q, fs=fs)
    candidates[f"iir_notch_q{int(q)}"] = {"b": b, "a": a, "type": "IIR", "delay_ms": 0.0}

# 4. Fungsi metrik
f_psd, P_in = welch(x, fs=fs, nperseg=min(4096, len(x)))

def band_power(P, f, lo, hi):
    m = (f >= lo) & (f <= hi)
    return float(np.trapezoid(P[m], f[m])) if np.any(m) else 0.0

def speech_distortion_db(y):
    f, P_y = welch(y, fs=fs, nperseg=min(4096, len(y)))
    m = (f >= spec["speech_band_hz"][0]) & (f <= spec["speech_band_hz"][1])
```

```

in_db = 10*np.log10(P_in[m] + 1e-18)
y_db = 10*np.log10(P_y[m] + 1e-18)
return float(np.sqrt(np.mean((y_db - in_db)**2)))

p_hum_in = band_power(P_in, f_psd, 45, 55)
metrics = {}
outputs = {}

for name, c in candidates.items():
    y = lfilter(c["b"], c["a"], x)
    outputs[name] = y
    f_y, P_y = welch(y, fs=fs, nperseg=min(4096, len(y)))
    p_hum_y = band_power(P_y, f_y, 45, 55)
    attenuation_db = 10*np.log10((p_hum_in + 1e-18)/(p_hum_y + 1e-18))
    dist_db = speech_distortion_db(y)
    # Direct-form estimate: numerator + feedback multiplies per sample
    mult = len(c["b"]) + max(0, len(c["a"])-1)
    if c["type"] == "FIR":
        delay_ms = float(c["delay_ms"])
    else:
        gd_f = np.linspace(spec["speech_band_hz"][0], spec["speech_band_hz"][1], 256)
        _, gd_samples = group_delay((c["b"], c["a"]), w=gd_f, fs=fs)
        gd_samples = gd_samples[np.isfinite(gd_samples)]
        delay_ms = float(np.percentile(np.abs(gd_samples), 95) / fs * 1000) if len(gd_samples) else 0.0
    meets = (
        attenuation_db >= spec["hum_attenuation_min_db"] and
        dist_db <= spec["speech_spectral_distortion_max_db_rms"] and
        delay_ms <= spec["additional_delay_max_ms"]
    )
    # quality dominates; compute cost is secondary
    score = dist_db + 0.01*mult + 0.05*delay_ms - 0.005*attenuation_db
    metrics[name] = {
        "hum_attenuation_db": float(attenuation_db),
        "speech_spectral_distortion_db_rms": float(dist_db),
        "estimated_multiplies_per_sample": int(mult),
        "additional_delay_ms": delay_ms,
        "meets_quality_requirements": bool(meets),
        "selection_score": float(score),
        "b_len": int(len(c["b"])),
        "a_len": int(len(c["a"])),
    }

eligible = [n for n in candidates if metrics[n]["meets_quality_requirements"]]
if eligible:
    selected = min(eligible, key=lambda n: metrics[n]["selection_score"])
else:
    # Fallback pedagogy: pilih skor terbaik dan tandai bahwa requirement belum terpenuhi
    selected = min(candidates, key=lambda n: metrics[n]["selection_score"])

final = candidates[selected]
y_final = outputs[selected]
peak = np.max(np.abs(y_final))
if peak > 0.999:
    y_final = y_final / peak * 0.98

# 5. Simpan audio final PCM16
wav_out = out_audio / "nyala_001_filter_final.wav"
wavfile.write(wav_out, fs, np.int16(np.clip(y_final, -1, 1) * 32767))

# 6. Simpan koefisien kandidat + final
npz_all = {}
for name, c in candidates.items():
    npz_all[f"{name}_b"] = np.asarray(c["b"], dtype=np.float64)
    npz_all[f"{name}_a"] = np.asarray(c["a"], dtype=np.float64)
np.savez(feat_dir / "mp06_filter_candidates.npz", **npz_all)
np.savez(feat_dir / "mp06_filter_final.npz",
    b=np.asarray(final["b"], dtype=np.float64),
    a=np.asarray(final["a"], dtype=np.float64),
    sample_rate_hz=np.array([fs]),
    selected_name=np.array([selected]))

# 7. Test vector untuk firmware (maks. 0,25 s)
N = min(len(x), int(0.25*fs))
test_x = x[:N]
test_y = lfilter(final["b"], final["a"], test_x)
np.savez(feat_dir / "mp06_filter_test_vector.npz",
    input=test_x.astype(np.float32),
    output=test_y.astype(np.float32),
    sample_rate_hz=np.array([fs]))

# 8. Export header C float32
b = np.asarray(final["b"], dtype=np.float32)
a = np.asarray(final["a"], dtype=np.float32)
def c_array(name, arr):
    vals = ", ".join(f"{v:.9g}f" for v in arr)

```

```

    return f"static const float {name}[{len(arr)}] = {{{vals}}};"
header = "\n".join([
    "#pragma once",
    "/* Auto-generated by Mini Project 6 */",
    f"#define AUDIO_FS_HZ {fs}",
    f"#define FILTER_B_LEN {len(b)}",
    f"#define FILTER_A_LEN {len(a)}",
    c_array("FILTER_B", b),
    c_array("FILTER_A", a),
    "/* Keep state between audio blocks; do not reset per frame. */",
])
with open(firmware / "mp06_filter_coefficients.h", "w", encoding="utf-8") as f:
    f.write(header + "\n")

# 9. Konfigurasi final + metrik
final_config = {
    "selected_filter": selected,
    "sample_rate_hz": int(fs),
    "b_length": int(len(final["b"])),
    "a_length": int(len(final["a"])),
    "input_audio": str(input_wav.relative_to(ROOT)),
    "output_audio": str(wav_out.relative_to(ROOT)),
    "coefficients": "features/intermediate/mp06_filter_final.npz",
    "firmware_header": "firmware/mp06_filter_coefficients.h",
    "test_vector": "features/intermediate/mp06_filter_test_vector.npz",
}
with open(config_dir / "filter_final_config.json", "w", encoding="utf-8") as f:
    json.dump(final_config, f, indent=2)

report = {"spec": spec, "selected_filter": selected, "candidates": metrics}
with open(out_met / "mp06_filter_design_metrics.json", "w", encoding="utf-8") as f:
    json.dump(report, f, indent=2)

# 10. Plot respons semua kandidat
fig, ax = plt.subplots(1, 1, figsize=(9, 5.5))
for name, c in candidates.items():
    f_r, H = freqz(c["b"], c["a"], worN=8192, fs=fs)
    ax.plot(f_r, 20*np.log10(np.abs(H)+1e-12), label=name, linewidth=1.0)
ax.axvline(50, linestyle="--", linewidth=0.8)
ax.set_xlim(0, 700); ax.set_ylim(-90, 5)
ax.set_xlabel("Frequency (Hz)"); ax.set_ylabel("Magnitude (dB)")
ax.set_title("Mini Project 6 - candidate filter responses")
ax.grid(alpha=0.2); ax.legend(fontsize=7)
plt.tight_layout(); plt.savefig(out_fig / "mp06_filter_candidates.png", dpi=180); plt.close()

# 11. Plot efek final pada PSD
f_y, P_y = welch(y_final, fs=fs, nperseg=min(4096, len(y_final)))
fig, ax = plt.subplots(2, 1, figsize=(9, 7))
show = min(len(x), int(0.2*fs)); t = np.arange(show)/fs
ax[0].plot(t, x[:show], label="input + hum", linewidth=0.8)
ax[0].plot(t, y_final[:show], label=f"final: {selected}", linewidth=0.8)
ax[0].set_xlabel("Time (s)"); ax[0].set_title("Waveform"); ax[0].legend(); ax[0].grid(alpha=0.2)
ax[1].plot(f_psd, 10*np.log10(P_in+1e-18), label="input")
ax[1].plot(f_y, 10*np.log10(P_y+1e-18), label="final")
ax[1].set_xlim(0, 1000); ax[1].set_xlabel("Frequency (Hz)"); ax[1].set_ylabel("PSD (dB/Hz)")
ax[1].set_title("PSD before / after final filter"); ax[1].legend(); ax[1].grid(alpha=0.2)
plt.tight_layout(); plt.savefig(out_fig / "mp06_filter_final_effect.png", dpi=180); plt.close()

# 12. Handoff ke MP7
status.update({
    "status": "ready_for_mp07",
    "last_completed_mini_project": 6,
    "filter_design_spec": "config/filter_design_spec.json",
    "filter_final_config": "config/filter_final_config.json",
    "filter_final_coefficients": "features/intermediate/mp06_filter_final.npz",
    "filter_test_vector": "features/intermediate/mp06_filter_test_vector.npz",
    "filter_final_audio": "data/processed/mp06_design/nyala_001_filter_final.wav",
    "filter_design_metrics": "results/metrics/mp06_filter_design_metrics.json",
    "mp07_input_audio": "data/processed/mp06_design/nyala_001_filter_final.wav",
})
with open(STATUS_PATH, "w", encoding="utf-8") as f:
    json.dump(status, f, indent=2)

print("Mini Project 6 selesai")
print("Selected filter:", selected)
for name in metrics:
    m = metrics[name]
    print(name, "atten=", round(m["hum_attenuation_db"], 2),
          "dist=", round(m["speech_spectral_distortion_db_rms"], 3),
          "mult=", m["estimated_multiplies_per_sample"],
          "pass=", m["meets_quality_requirements"])
print("Status: ready_for_mp07")

```

Langkah 3. Jalankan Skrip

```
python mini_project_06.py
```

Terminal harus menampilkan nama filter terpilih, metrik setiap kandidat, dan Status: ready_for_mp07. Filter terpilih dapat berbeda jika rekaman masukan berbeda, karena seleksi memang menggunakan metrik aktual pada audio proyek.

Langkah 4. Interpretasikan Keputusan Desain

Buka results/figures/mp06_filter_candidates.png dan identifikasi perbedaan selective notch, high-pass FIR, dan high-pass IIR. Setelah itu buka mp06_filter_final_effect.png. Pastikan gangguan sekitar 50 Hz turun tanpa perubahan berlebihan pada pita 150–4000 Hz. Buka results/metrics/mp06_filter_design_metrics.json untuk melihat kandidat mana yang memenuhi requirement dan mengapa satu desain dipilih.

Langkah 5. Validasi Artefak dan Handoff ke Mini Project 7

Pastikan seluruh artefak pada tabel berikut tersedia. Buka firmware/mp06_filter_coefficients.h dan pastikan array koefisien telah dibuat. Jangan mengubah koefisien secara manual setelah file ini terbentuk; bila spesifikasi berubah, jalankan kembali skrip dari sumber konfigurasi.

Tabel 6.11. Kriteria lulus Mini Project 6 dan handoff ke Mini Project 7.

Artefak / pemeriksaan	Kriteria lulus	Digunakan pada MP7
project_status.json	status = ready_for_mp07	Mengaktifkan Mini Project 7
filter_design_spec.json	Acceptance criteria tersimpan	Dokumentasi preprocessing
filter_final_config.json	Nama dan parameter filter final jelas	Konfigurasi pipeline
mp06_filter_final.npz	Koefisien final tersedia	Reproduksi di Python
mp06_filter_coefficients.h	Header C dapat dibuka	Persiapan embedded
mp06_filter_test_vector.npz	Input/output referensi tersedia	Validasi lintas platform
nyala_001_filter_final.wav	Audio dapat diputar dan tidak clipping	Input preprocessing
mp06_filter_design_metrics.json	Semua kandidat dan selected_filter tercatat	Audit keputusan
mp06_filter_candidates.png	Respons kandidat terbaca	Bukti perbandingan
mp06_filter_final_effect.png	Efek final terlihat pada waveform/PSD	Bukti kualitas

Aturan serial

Mini Project 7 harus membaca mp07_input_audio dari project_status.json. Jangan kembali menggunakan audio MP4 atau MP5, karena mulai titik ini filter yang dipilih pada MP6 menjadi bagian resmi pipeline preprocessing.

Referensi Bab 6

1. Oppenheim, A. V., & Schafer, R. W. Discrete-Time Signal Processing. Pearson.

2. Lyons, R. G. Understanding Digital Signal Processing. Pearson.
3. Zölzer, U. Digital Audio Signal Processing. Wiley.
4. Mitra, S. K. Digital Signal Processing: A Computer-Based Approach. McGraw-Hill.
5. Parks, T. W., & McClellan, J. H. Chebyshev Approximation for Nonrecursive Digital Filters with Linear Phase.
6. Proakis, J. G., & Manolakis, D. G. Digital Signal Processing: Principles, Algorithms, and Applications. Pearson.
7. SciPy Signal documentation: FIR and IIR filter design and analysis routines.
8. ARM. CMSIS-DSP Software Library documentation for FIR and biquad filters.
9. Espressif Systems. ESP-DSP library documentation.